

Core Concept 3: Session Rotation & Proxy Architecture

When a script asks a website for 100 pages of data in 5 seconds, the website immediately knows it is a bot. The consequence is an **IP Ban** or a **Session Ban**.

To build a scraping infrastructure that pulls 100,000 records daily, you cannot rely on one script and one laptop. You must use Rotation.

The Hierarchy of Identity

Websites track you using three primary identifiers:

1. **IP Address (The "Where")**: Your router's public address.
2. **Browser Fingerprint (The "What")**: Playwright vs Chrome vs Firefox, screen resolution, OS.
3. **Session Cookies (The "Who")**: The specific `session_id` tokens given by the server.

If *any* of these three vectors look suspicious, you get blocked.

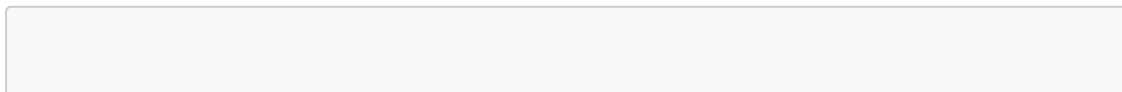
Architecture 1: Session Rotation (The Cheap Way)

As you correctly identified, rotating IPs is expensive. Sometimes, websites flag your session cookies simply because they hit an artificial "time limit" (e.g., 50 pages). Your IP isn't banned yet; they just want you to reset.

The Strategy

1. Open Playwright on your Home IP.
2. Get Session A cookies.
3. Pass Session A to `aiohttp`.
4. Scrape 50 records. Backend throws a `403 Forbidden`.
5. **DO NOT GET A NEW IP.**
6. Instead: Delete Session A from memory.
7. Open a *new* Playwright context (this clears all cookies).
8. Get Session B cookies.
9. Pass Session B to `aiohttp`.
10. Scrape records 51-100.

Code Example: The Session Manager Loop



```

async def resilient_scraper(start_url, max_failures=3):
    current_page = 1
    failures = 0

    # 1. Boot up the "Key Maker" to get our first session
    headers = get_golden_tickets(start_url)

    async with aiohttp.ClientSession() as session:
        while current_page < 1000 and failures < max_failures:

            payload = {"pg_no": current_page}
            try:
                # 2. Try the fast API fetch
                async with session.post(API_URL, json=payload,
headers=headers) as response:

                    if response.status == 200:
                        data = await response.json()
                        save_data(data)
                        current_page += 1
                        failures = 0 # Reset failure count on
success

                    elif response.status in [401, 403, 429]:
                        print("🔥 Session Burned! We hit the
behavioral cap.")

                        # 3. The magic step: Rotate the Session!
                        print("🔄 Generating brand new session
cookies...")

                        headers = get_golden_tickets(start_url)
                        failures += 1

            except Exception as e:
                failures += 1

            await asyncio.sleep(2) # Sleep to avoid rate limiting

```

Architecture 2: IP Rotation / Proxies (The Professional Way)

Eventually, no matter how many times you rotate your cookies, the server will notice that 5,000 requests are coming from 192.168.1.5 in Delhi. At that point, your IP is hard-banned. The website won't even load in your normal browser.

To solve this, businesses rent **Residential Proxies** (e.g., BrightData, Oxylabs, Smartproxy). These services route your Python script's traffic through the actual laptops and cellphones

of regular people around the world.

Proxy Integration in Python

Instead of sending requests from your machine, you send them through the Proxy Provider's gateway.

```
# The proxy gateway provided by your paid service
PROXY_URL = "http://username:password@gate.smartproxy.com:7000"

async def fetch_with_proxy(url, proxy_url):
    # Pass the proxy to the aiohttp session
    async with aiohttp.ClientSession() as session:
        # Every time aiohttp sends this GET request, the Proxy
        # Provider
        # dynamically assigns a brand new, random IP address to
        # it.
        async with session.get(url, proxy=proxy_url) as response:
            return await response.text()
```

The Ultimate Architecture: The Distributed Job Queue

If you need 1,000,000 records, you do not use a `for` loop. You use a Database and a Task Queue (like Redis + Celery).

1. **The Planner:** A script pushes 10,000 targeted URLs (e.g., "Delhi Tech Shops", "Mumbai Plumbers") into a Redis Queue.
2. **The Workers:** You rent 5 cheap Linux servers. Each server runs 10 parallel Python scripts.
3. **The Execution:**
 - Worker 1 pops "Delhi Tech Shops" from the Queue.
 - Worker 1 grabs a Random IP from the Proxy Pool.
 - Worker 1 launches Playwright (Hybrid method) to get valid Cookies.
 - Worker 1 uses `aiohttp` to scrape 50 records.
 - Worker 1 uploads the 50 records to a central PostgreSQL database.
 - Worker 1 destroys the session, asks Redis for the next URL, and repeats.

This is how multi-million dollar data businesses function. They treat scraping as a horizontally scalable, distributed micro-service.